Student 11 **KAVI PREETHA G** 192420040RC 20 / 20 submitted

Q1. Void Function (No Return Value)

CODE

```
def welcome():
    print("Welcome to Python Programming")
welcome()
```

OUTPUT

```
(no output)
```

Q2. Void Function (No Return Value)

CODE

```
def print_table(num):
    for i in range(1, 11):
        print(f"{num} * {i} = {num * i}")

n = int(input())
print_table(n)
```

INPUT

```
5
```

OUTPUT

```
(no output)
```

Q3. Void Function (No Return Value)

CODE

```
def print_evens(n):
    for i in range(1, n+1):
        if i % 2 == 0:
            print(i)

num = int(input())
print_evens(num)
```

INPUT

```
10
```

OUTPUT

```
(no output)
```

Q4. Fruitful Function (Returns One Value)

CODE

```
def count_vowels(s):
    count = 0
    for char in s.lower():
        if char in "aeiou":
            count += 1
    return count

text = input()

print(count_vowels(text))
```

INPUT

```
hello
```

OUTPUT

```
(no output)
```

Q5. Fruitful Function (Returns One Value)

CODE

```
def find_largest(nums):
    return max(nums)
user_list = [int(x) for x in input().split()]

print(find_largest(user_list))
```

INPUT

```
12 45 7 89 23
```

OUTPUT

```
(no output)
```

Q6. Fruitful Function (Returns One Value)

CODE

```
def check_prime(n):
    if n ≤ 1:
        return "Not Prime"
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return "Not Prime"
    return "Prime"
num = int(input())
print(check_prime(num))
```

INPUT

```
11
```

OUTPUT

```
(no output)
```

Q7. Function Returning Two or More Values

CODE

```
def process_marks(m1, m2, m3):
    total = m1 + m2 + m3
    avg = total / 3
    if avg ≥ 90:
        grade = "A"
    elif avg ≥ 80:
        grade = "B"
    elif avg ≥ 70:
        grade = "C"
    else:
        grade = "D"
    return total, round(avg, 2), grade
s1 = int(input())
s2 = int(input())
s3 = int(input())
tot, average, grd = process_marks(s1, s2, s3)
print(tot)
print(average)
print(grd)
```

INPUT

```
90
85
80
```

OUTPUT

```
(no output)
```

Q8. Function Returning Two or More Values

CODE

```
def circle_calc(r):

    area = 3.14 * r * r
    circumference = 2 * 3.14 * r
    return round(area, 2), round(circumference, 2)

radius = float(input())
a, c = circle_calc(radius)
print(a)
print(c)
```

INPUT

```
5
```

OUTPUT

```
(no output)
```

Q9. Function Returning Two or More Values

CODE

```
def calc_values(n):
    sq = n * n
    cb = n * n * n
    sr = n ** 0.5
    return sq, cb, round(sr, 2)

num = int(input())
s, c, r = calc_values(num)

print(s)
print(c)
print(r)
```

INPUT

```
4
```

OUTPUT

```
(no output)
```

Q10. Keyword Arguments

CODE

```
def student(name, age, department):
    print(name)
    print(age)
    print(department)

n = input()
a = input()
d= input()

student(department=d, age=a, name=n)
```

INPUT

```
Kavi Preeetha
19
Engineering
```

OUTPUT

```
(no output)
```

Q11. Keyword Arguments

CODE

```
def employee(empid, name, salary):
    print(empid)
    print(name)
    print(salary)

i = input()
n = input()
s = input()


employee(name=n, salary=s, empid=i)
```

INPUT

```
101
Joshua
50000
```

OUTPUT

```
(no output)
```

Q12. Keyword Arguments

CODE

```
def book(title, author, price):
    print(title)
    print(author)
    print(price)

t = input()
a = input()
p = input()

book(title=t, author=a, price=p)
```

INPUT

```
Python Programming
John Smith
450
```

OUTPUT

```
(no output)
```

Q13. Variable-Length Arguments (*args)

CODE

```
import sys

def add_numbers(*args):
    return sum(args)

inputs = sys.stdin.read().split()
nums = [int(x) for x in inputs]

print(add_numbers(*nums))
```

INPUT

```
10 20 30 40
```

OUTPUT

```
(no output)
```

Q14. Variable-Length Arguments (*args)

CODE

```
import sys

def calc_avg(*args):
    return sum(args)/len(args) if args else 0

nums = [int(x) for x in sys.stdin.read().split()]
res = calc_avg(*nums)

print(int(res) if res.is_integer() else round(res, 2))
```

INPUT

```
80 90 100
```

OUTPUT

```
(no output)
```

Q15. Variable-Length Arguments (*args)

CODE

```
import sys

def print_strings(*args):
    for s in args:
        print(s)

words = sys.stdin.read().split()

print_strings(*words)
```

INPUT

```
apple banana cherry
```

OUTPUT

```
(no output)
```

Q16. Variable-Length Arguments (*args)

CODE

```
import sys

def get_largest(*args):
    return max(args) if args else 0
nums = [int(x) for x in sys.stdin.read().split()]

print(get_largest(*nums))
```

INPUT

```
12 45 7 89 23
```

OUTPUT

```
(no output)
```

Q17. Recursive Functions

CODE

```
def fib(n):
    if n ≤ 1:
        return n
    return fib(n-1) + fib(n-2)
n = int(input())
for i in range(n):
        print(fib(i))
```

INPUT

```
5
```

OUTPUT

```
(no output)
```

Q18. Recursive Functions

CODE

```
def sum_digits(n):
    if n == 0:
        return 0
    return (n % 10) + sum_digits(n // 10)
num = int(input())

print(sum_digits(num))
```

INPUT

```
1234
```

OUTPUT

```
(no output)
```

Q19. Recursive Functions

CODE

```
def rev(s):
    if len(s) == 0:
        return s
    return rev(s[1:]) + s[0]
text = input()

print(rev(text))
```

INPUT

```
hello
```

OUTPUT

```
(no output)
```

Q20. Recursive Functions

CODE

```
def power(x, y):
    if y ==0:
        return 1
    return x * power(x, y -1)

base = int(input())
exp = int(input())

print(power(base, exp))
```

INPUT

```
2
3
```

OUTPUT

```
(no output)
```